XINGQIANG CHEN

Hangzhou, China
+86 150-0133-4125 ◇ joy6677@outlook.com ◇ github.com/chenxingqiang

OBJECTIVE

Looking for **FULL TIME** roles in **AI Accelerator Compiler / System Software** · **LLM Inference Systems** · **HW/SW Co-design**.

INTERDISCIPLINARY BACKGROUND

Physics and biophysics training across computational chemistry, molecular dynamics, and computer science. Contests: **CUMCM National First Prize** (2012); **MCM Meritorious Winner** (2013); **Second Prize**, CCF Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (2025).

EDUCATION

Doctoral Studies in Biophysics and Condensed Matter Sep. 2014 - Jul. 2018
Xiamen University, Department of Physics and Engineering; protein dynamics and QM/MM simulation
Certificate of completion; degree defense in progress

B.S. in Theoretical Physics Sep. 2010 - Jul. 2014
Xiamen University, Department of Physics and Engineering

SKILLS AND PROJECTS

Languages: C++, Python (primary); CUDA, Java, Scala, Go.

Compiler / Toolchain: MLIR, LLVM backends and ISA bring-up, TVM, graph partitioning, operator fusion, cost modeling, codegen, auto-tuning.

Chip Software / Verification: Palladium emulation, GDB co-debug against simulation models, firmware and weight-map generation, operator-level metrics (Cycles/MACs).

Inference Systems: Transformers, KV-Cache, quantization, batching and scheduling, Triton kernels, FlashAttention, CUPTI/Perfetto profiling.

Platforms: PyTorch, TensorFlow, ONNX, Kubernetes; Spark, Flink, Hadoop.

Open Source & Releases (`github.com/chenxingqiang` unless noted):

- **TraceSmith** — GPU profiling & replay across CUDA/ROCm/Metal/MACA/Ascend; C++ core, Python bindings, on PyPI. `TraceSmith`
- **YiRage Persistent Kernel** — LLM megakernel compiler and runtime, multi-backend. `YiRage`
- **JAX-sklearn** — sklearn-compatible API on JAX; 13,058 tests passing; on PyPI. `jax-sklearn`
- **Secret-Learn** — Privacy-preserving ML: federated learning, secret sharing, split learning. `Secret-Learn`
- **YouTube-DNN-RecSys** — TensorFlow + Spark retrieval and ranking (60 stars). `YouTube-DNN-RecSys`
- **Research CLI** — Command-line AI tooling for research workflows. `iechor-research/research-cli`
- **Questions-Gen** — Math-problem generation (Qwen3-14B, GRPO, distillation); CLI and HF weights. `GenQuestion`
- **Architecture appendix** — System architecture diagrams for the 7 projects above (US Letter, same page size as this resume; see `docs/architecture/`).

EXPERIENCE

Compiler & Toolchain Expert, Witmem Technology (ByteDance) Hangzhou, China
Compiler / Palladium Verification *Dec. 10, 2025 - Present*

- Owned the product-line compiler and Palladium emulation for a CIM chip: model compile, codegen, on-chip execution, and defect isolation on the emulator.

- Brought up software on Palladium: ran compiler output on the emulator and jointly debugged with GDB across the simulation model, silicon, and system software.
- Implemented **CIM-MLIR**: PyTorch/TensorFlow/ONNX lowered via TOSA/StableHLO to CPU, GPU, and CIM, covering frontend, graph partition, hardware backend, and runtime.
- Implemented cost-driven graph partition and CIMHigh/Mid/Low IR; used OR-Tools for Die/L2/L0 allocation; emitted burnable **config.bin**, weight **map.bin**, GPU PTX, and LLVM host code.
- Implemented a cross-device runtime: per-device executors, unified memory, compute/transfer overlap and triple buffering; C++/Python APIs with latency, utilization, and transfer-cost counters.
- Implemented **GRC-PIM**: drift-gated, GEMV-reusing residual calibration and soft-decision error correction for PIM; submitted to *npj Unconventional Computing* (see Publications).

Senior AI Compiler Engineer, Yizhu Tech Ltd.
Hangzhou, China
R&D Department *Sep. 2022 - Dec. 2025*

- Led **YiRage Persistent Kernel (YPK)**, compiling end-to-end LLM decode into a single megakernel; **1.2×–6.7×** end-to-end latency gain over a PyTorch eager baseline on Llama2-7B, single A100 (varies with batch and sequence length).
- Implemented fused operators (RMSNorm+Linear, GQA) and a persistent execution model, cutting kernel-launch and intermediate-tensor round trips; largest gains on the batch=1 decode path.
- Built the scheduling layer and integrated Triton and FlashInfer kernels; multi-GPU scaling validated up to **8** devices with NVSHMEM-backed tensors.
- Developed graph compilation and auto-tuning: kernel parameter search and KV-cache-friendly memory layout to reduce memory traffic.
- Filed **2** invention patents; delivered the stack into **2** datacenter customer PoCs.
- Owned the LLVM backend Target and complex-ISA support for the in-house SoC, driving backend standardization and the software ecosystem.
- Built model-conversion and operator-level evaluation: integrated TVM, ONNX, MLCommons, TensorFlow/PyTorch into **YZlite**, reporting Cycles/MACs and multi-chip benchmarks. `ReRAM-Labs/yzlite`

Privacy-preserving Algorithms Engineer, Trustbe
Hangzhou, China
IT Group *Jan. 2022 - Sep. 2022*

- Built a homomorphic-encryption federated learning platform for cross-institution modeling in finance, improving model quality over local-only baselines while keeping privacy constraints.
- Designed MPC protocols for joint training and inference of bank risk models, balancing protocol security against communication cost.
- Open-sourced **Secret-Learn**: sklearn-compatible API across federated learning, secret sharing, and split learning, JAX-accelerated and integrated with SecretFlow; on PyPI. `Secret-Learn`
- Filed **3** invention patents, all published: `CN114925786A`, `CN114997345A`, `CN115130681A`.

Staff Algorithms Engineer, BioMap
Beijing, China
Algorithm Engineering Group *Sep. 2021 - Dec. 2021*

- Built bioinformatics tooling and a unified data platform for large-scale sequence analysis, structure prediction, and functional annotation.
- Set up evaluation pipelines for protein structure prediction models (AlphaFold family) feeding downstream drug design.
- Two registered software copyrights (2021SR2004464, 2021SR1862397); open-sourced AlphaFold2 code walkthroughs (26 stars). `alphafold2-codecs`

Senior Algorithms Engineer, Yalla Group (NYSE: YALA)
Hangzhou, China
Data Center Engineering Group *Sep. 2020 - Sep. 2021*

- Rebuilt the ChatRooms recommender with multi-objective ranking; A/B tests showed significant gains in daily active time and 7-day retention (**42% → 61%**).
- Built Ludo risk control from real-time behavior features and ML models, materially reducing fraudulent refunds.
- Developed a card-game AI with deep RL for human-vs-bot play and matchmaking fill, improving off-peak match quality and retention.

Senior Algorithms Engineer, Gejia Network Tech — Hangzhou, China
Search & Recommendation Engineering Group — *Mar. 2019 - Sep. 2020*

- Designed multi-scenario ranking with collaborative filtering, deep ranking models, and a product knowledge graph; online experiments delivered double-digit relative CTR and CVR gains.
- Optimized the serving path with feature caching and candidate pruning, holding P99 latency under **50ms** through peak promotional traffic.
- Open-sourced YouTube-DNN-RecSys, a TensorFlow + Spark retrieval and ranking reference (60 stars). `chenxingqiang/YouTube-DNN-RecSys`

Machine Learning Engineer, Bianlifeng Tech — Beijing, China
Data Engineering Group — *Mar. 2017 - Mar. 2019*

- Built store-level sales forecasting across **50+** stores using gridded weather and spatiotemporal features, reducing stock-outs and dead stock while improving turnover.
- Linked customer identities to purchase history over tens of millions of events to power personalized recommendations.
- Shipped a product recognition model for **500+** SKUs used in fast checkout, cutting checkout and queue time.

RESEARCH & INTERNSHIPS

Molecular Dynamics Simulation of Proteins with QM/MM model — Xiamen university, China
(China National Funds) — *Sep. 2014 - Mar. 2017*

- Ran QM/MM protein dynamics simulations and conformational analysis; results published in Acta Cryst. D, PLoS ONE, and Phys. Rev. E (see Publications).

Liquid Biopsy ct-DNA Kit and DNA Methylation Analysis — Lepu Medical, China
(R&D Engineer Intern) — *Mar. 2016 - Mar. 2017*

- Analyzed roughly 10,000 public gene-chip samples (TCGA/GEO) to localize Septin9 methylation sites, an early colon-cancer marker, informing biomarker selection for a clinical diagnostic kit.

AWARDS & SERVICE

- **National First Prize**, China Undergraduate Mathematical Contest in Modeling (CUMCM), 2012.
- **Meritorious Winner**, Mathematical Contest in Modeling (MCM/COMAP), 2013.
- **6th** worldwide, International Solar Decathlon Competition, 2013.
- Ranks **57** and **32** of 1739 teams, IEEE ICDM Global Meteorological AI Challenge, 2018.
- **Second Prize**, 8th CCF Open Source Innovation Competition — Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (CCF/ODTC), 2025. CCF announcement
- Judge, first screening, Best Practices of Applied Algorithms, 2021.

PUBLICATIONS

- Chen X. GRC-PIM: A Drift-Gated, GEMV-Reusing Residual Calibrator for Soft-Decision Error Correction in Processing-in-Memory. *npj Unconventional Computing* (submitted).
- Cai Y, Su M, Ahmad A, Hu X, Sang J, Kong L, Chen X, Wang C, Shuai J, Han A. Conformational dynamics of the essential sensor histidine kinase WalK. *Acta Crystallographica Section D: Structural Biology.* 2017 Oct 1;73(10):793-803.
- Xia H, Chen X, Chen X, Cheng J, Liu Y, Chen X, Zhang Q, Zou G. Chiral discrimination of amino acid enantiomers based on different interactions with Cu2+. *Sensors and Actuators B: Chemical.* 2018 Jan 1;254:44-51.
- Ahmad A, Cai Y, Chen X, Shuai J, Han A. Conformational dynamics of response regulator RegX3 from Mycobacterium tuberculosis. *PloS one.* 2015 Jul 22;10(7):e0133389.
- Chen Y, Qi H, Li X, Cai M, Chen X, Liu W, Shuai J. Suppressing effect of C a 2+ blips on puff amplitudes by inhibiting channels to prevent recovery. *Physical Review E.* 2016 Aug 15;94(2):022411.

Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · US Letter, same page size as the resume

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

READING GUIDE

Dashed frames are architecture zones, solid inner frames are subsystems, and boxes are concrete modules.
Each page states which parts the system contains and where their responsibility boundaries lie.

CONTENTS

TraceSmith
GPU profiling & replay

YiRage
RuntimeFusion

JAX-sklearn
transparent JAX acceleration

Secret-Learn
FL · SS · SL

YouTube-DNN
candidate generation

Research CLI
agent workflow

Questions-Gen
three-stage training

TraceSmith — GPU Profiling & Replay Architecture

Applications, pybind11 binding, C++17 core, GPU backends, and output formats

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

TraceSmith is a cross-platform GPU profiling and deterministic replay system for AI compiler, framework, and driver engineers.
Its architecture separates applications, the pybind11 binding, a C++17 core, GPU backends, and output formats.

APPLICATION · user entry

- **C++ Application** — in-process capture
- **Python Application** — in-process capture
- **C++ CLI** — tracesmith profile · record · view
- **Python CLI** — tracesmith-cli export · analyze · replay
- **External UI** — Perfetto UI Chrome tracing

BINDINGS · pybind11 boundary

- **_tracesmith.so** — python/src/bindings.cpp pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — cross-device capture
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — collective ops
- **ClockCorrelator** — clock alignment
- **CommAnalysis** — communication matrix

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU memory
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer xctrace · msprof

State Module

- **GPUStateMachine** — state analysis
- **InstructionStream** — dependency graph
- **TimelineBuilder** — ordered events
- **TimelineViewer** — ASCII rendering
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

TraceSmith — GPU Profiling & Replay Architecture

Applications, pybind11 binding, C++17 core, GPU backends, and output formats

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

CORE LIBRARY · C++17 · continued

Replay Module

- **ReplayEngine** — core orchestrator
- **StreamScheduler** — operation order
- **OperationExecutor** — launch replay
- **DeterminismChecker** — result compare
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — binary writer
- **SBTReader** — binary reader
- **.sbt** — header + events counters

Configuration

- **ProfilerConfig** — buffer · overflow
- **ReplayConfig** — mode · speed
- **StackCaptureConfig** — stack policy
- **NCCLTrackerConfig** — collective policy

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — breakpoints · replay
- **tracesmith-gdbserver** — v0.10.0

GPU BACKENDS · system interfaces

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — distributed
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — host stacks
- **Perfetto SDK** — trace export

TraceSmith — GPU Profiling & Replay Architecture

Applications, pybind11 binding, C++17 core, GPU backends, and output formats

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility



The C++17 core contains capture, state, format, replay, cluster, configuration, and GDB. Applications, bindings, backends, and outputs are its boundaries.

YiRage (YPK) — RuntimeFusion Architecture

Inference engine, RuntimeFusion, search plan, backend manager, implementations, and hardware

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion.

The architecture separates the inference engine, RuntimeFusion, search plans, the backend manager, backend implementations, and hardware targets.

INFERENCE ENGINE · scheduling owner

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up + down
- **Native Fallback** — engine native operators

SEARCH & PLAN · offline / warm-up

- **superoptimize** — KN / TB graph search
- **FusionPlan** — µGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API & BACKEND MANAGER

- **KNGraph / TBGraph** — kernel transpiler
- **BackendRegistry** — search + execute same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — optional

BACKEND IMPLEMENTATIONS

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

The engine owns scheduling and KV. RuntimeFusion owns FusionCapsules. BackendRegistry binds search and execution to the same backend.

JAX-sklearn (xlearn) — Transparent Acceleration Architecture

User API, compatibility layer, JAX implementations, data policy, and hardware backends

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

JAX-sklearn provides a drop-in scikit-learn API while transparently accelerating supported estimators with JAX. The architecture has a user API, a compatibility layer, JAX implementations, a data policy, and hardware backends.

USER CODE · sklearn-compatible API

- **import xlearn as sklearn** — 100% call surface
- **Estimators** — linear_model · cluster · decomposition · preprocessing
- **Calls** — fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__** — lazy submodule import
- **_auto_jax_accelerate_module** — replace estimator classes
- **AcceleratorRegistry** — _accelerator.py
- **create_intelligent_proxy** — _proxy.py
- **monkey_patch_estimator** — install proxy class

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy** — fit-time dispatch
- **create_accelerated_estimator** — _accelerator.py
- **_universal_jax.py** — JIT · vmap
- **original_class** — sklearn fallback
- **_gp.py** — Gaussian process
- **_mlp.py** — neural net
- **_svm.py** — SVM kernels
- **_tree.py** — tree models

DATA MANAGEMENT · arrays and policy

- **_data_conversion.py** — NumPy ↔ JAX · dtype · shape
- **_config.py** — enable_jax · jax_platform
- **jax_auto_threshold** — complexity >= 1e8
- **detect_hardware** — install command · platform probe

HARDWARE · JAX extras

- **CPU** — jax-cpu · jaxlib
- **CUDA GPU** — jax-gpu · jax-cuda11
- **TPU** — jax-tpu
- **Apple Metal** — jax-metal · float32 · experimental

The compatibility layer places EstimatorProxy between the sklearn API and either a JAX implementation or the original estimator.

Secret-Learn — Privacy-Preserving ML Architecture

Applications, sklearn API, FL / SS / SL modes, JAX acceleration, secure devices, and infrastructure

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

Secret-Learn exposes sklearn-compatible machine learning across federated, secret-sharing, and split-learning modes.

The architecture comprises the sklearn API, three privacy modes, JAX acceleration, SecretFlow devices, and infrastructure.

APPLICATION · multi-party use cases

- **Healthcare** — hospital collaboration, patient privacy
- **Finance** — cross-bank analysis, fraud detection
- **IoT / Edge** — edge intelligence, distributed learning
- **Research** — genomics, multi-institution
- **Government** — census analytics, cross-agency data

SKLEARN API · 191 algorithms

- **Linear Models** — 39 — Ridge · Lasso · SGD
- **Ensemble** — 18 — RF · XGBoost
- **Clustering** — 14 — KMeans · DBSCAN
- **Decomposition** — 14 — PCA · NMF · ICA
- **SVM** — 7 — SVC · SVR
- **Neural Networks** — 4 — MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees, Naive Bayes

PRIVACY MODES · execution placement

FEDERATED LEARNING

- **Local PYU** — data stays local
- **HEU Aggregate** — secure averaging
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING

- **SPU MPC** — ciphertext compute
- **ABY3 / CHEETAH** — MPC protocols
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING

- **Model Split** — across parties
- **HEU Activations** — protected tensors
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

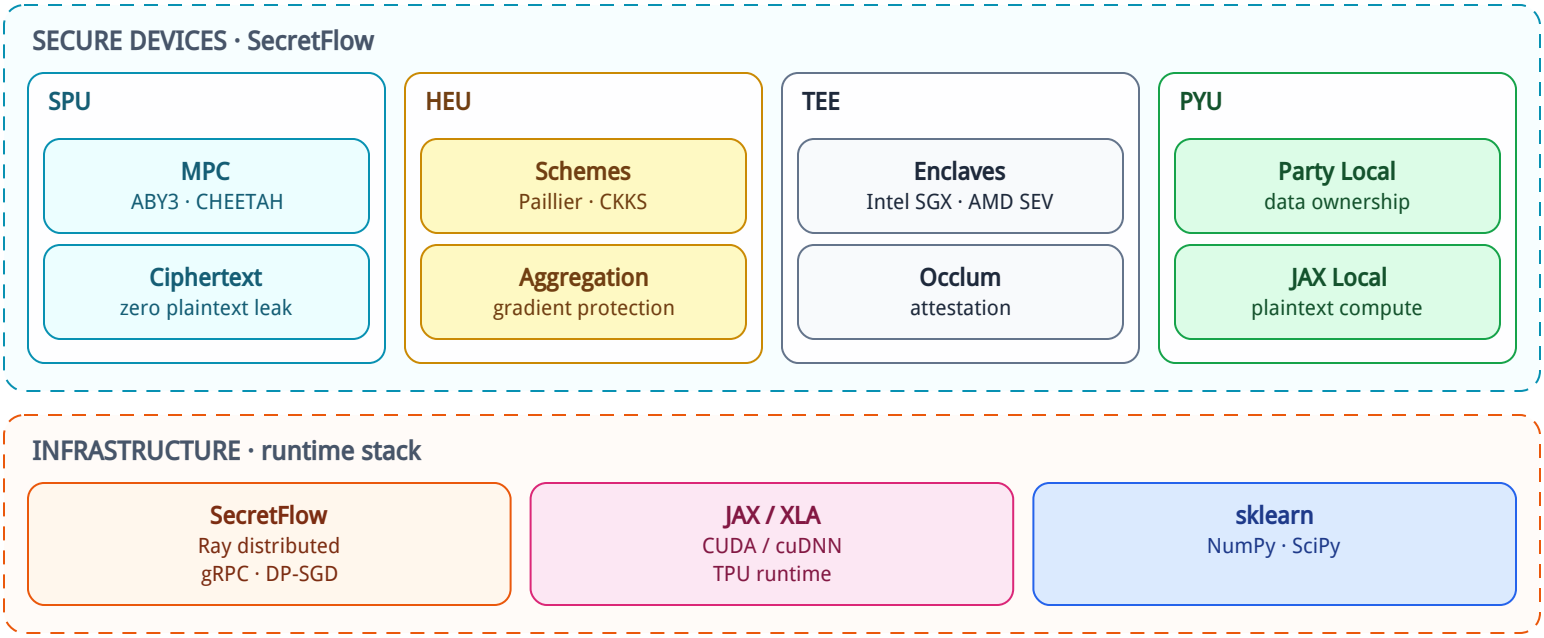
JAX ACCELERATION · xlearn bridge

- **JIT / XLA** — traced execution
- **jax.vmap** — batching · autodiff
- **xlearn Backend** — JAX-sklearn bridge
- **Smart Fallback** — original sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

Secret-Learn — Privacy-Preserving ML Architecture

Applications, sklearn API, FL / SS / SL modes, JAX acceleration, secure devices, and infrastructure

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility



FL, SS, and SL are three placements of the same sklearn API. JAX and SecretFlow devices PYU, HEU, SPU, and TEE form the compute boundary.

YouTube-DNN-RecSys — Candidate Generation Architecture

Data processing, storage, dnn.py model, prediction services, and support utilities

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

YouTube-DNN-RecSys is a TensorFlow and Spark implementation of the candidate-generation stage of YouTube recommendations.
The architecture comprises data processing, storage, the dnn.py model, prediction services, and support utilities.

DATA PROCESSING · offline sample build

Python Pipeline

data2tfrecords.py
random training data

TFRecords
train / evaluation

read_tfrecords.py
dataset reader

Spark Pipeline

MakeDataOne.scala
distributed generation

MakeDataTwo.scala
TFRecords output

HDFS
dataset storage

STORAGE · datasets and checkpoints

data/tfrecords/
train/train.tfrecords
evaluation/evaluation.tfrecords

data/checkpoints/
ckpt/checkpoint
ckpt/eval · modelpath/

MODEL TRAINING · dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 items × 16

Average Pooling
history vector

DNN Tower
Dense 64 + ReLU
Dropout · Dense 32 + ReLU
user vector 16

Softmax Output
500 classes
cross entropy · Top-K

PREDICTION · serving paths

PredictUserVector.scala
SavedModel
user vector head

ItemEmbeddingPredictor.scala
item embedding table
FeatureBuilder

load_dnn_model.py
SavedModel load
online / batch inference

UTILITIES · training support

tensor_board.py
TensorBoard

feature_column.py
feature spec

self_defined_network_layer.py
custom TF layers

Spark Base
BaseSparkLocal.scala
BaseSparkOnline.scala

YouTube-DNN-RecSys — Candidate Generation Architecture

Data processing, storage, dnn.py model, prediction services, and support utilities

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

UTILITIES · training support · continued

Configuration
vars.prod.properties
vars.sit.properties

The model contains input, item embedding, average pooling, the user tower, and softmax. Python and Spark each provide data and prediction components.

Research CLI — Agent System Architecture

CLI, core session and routing, tools and research modules, model providers, and embedded runtimes

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW

Research CLI is a terminal-first AI agent for code, tools, MCP services, and academic research workflows.
The architecture separates the CLI, core session and routing, tools and research modules, model providers, and embedded runtimes.

CLI FRONTEND · packages/cli

- **research.tsx** — interactive TUI
- **commands/** — mcp.ts · skills.tsx · hooks.tsx
- **ui/commands/research** — arxivCommand · paper · investigate · submit
- **acpSession.ts** — ACP session · stdio transport

CORE ORCHESTRATION · packages/core

Session

- **researchChat.ts** — researchRequest.ts
- **research-config-manager.ts** — academic settings
- **config / memory** — context window

Routing

- **modelRouterService** — routing/modelRouterService.ts
- **defaultStrategy** — routingStrategy.ts
- **classifierStrategy** — gemma · numerical
- **compositeStrategy** — approval · fallback · override

TOOLS & RESEARCH MODULES

Workspace

- **edit.ts · grep.ts** — filesystem changes
- **shell / ls** — local execution

MCP

- **mcp-client.ts** — server sessions
- **mcp-client-manager.ts** — resource routing

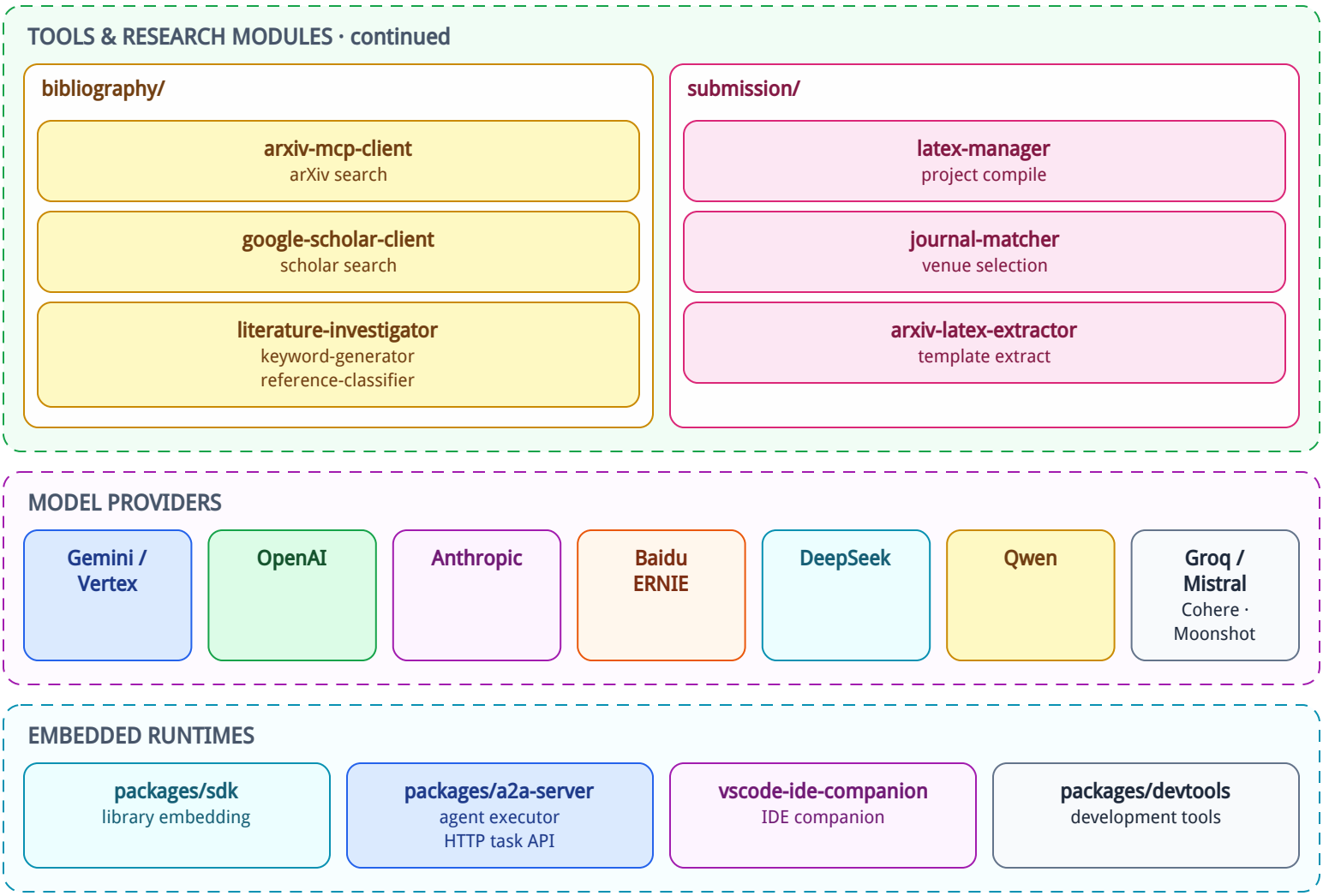
analysis/

- **bibliography-manager** — citations
- **experiment-code-generator** — Python · R · Julia
- **research-data-analyzer** — statistics · ML

Research CLI — Agent System Architecture

CLI, core session and routing, tools and research modules, model providers, and embedded runtimes

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility



The CLI and ACP form the frontend. Core owns session and model routing. Tools, research modules, providers, and embedded runtimes are separate boundaries.

Questions-Gen — Three-Stage Training Architecture

Training corpus, SFT, GRPO, distillation, validation, and deployment interfaces

Dashed frame = architecture zone · Solid frame = subsystem · Box = module and responsibility

PROJECT OVERVIEW
Questions-Gen is a Qwen3-14B system for generating and evaluating mathematical competition problems.
The architecture contains the training corpus, SFT, GRPO, distillation, validation, and deployment interfaces.

TRAINING DATA · documented mixture

- **Historical** — 50% — QuestionDataPreparer.py
- **Conditional Variants** — 30% — data_preparer.py
- **Innovative Types** — 20% — real_math_data_processor.py

Stage 1 · SFT

- **Qwen3-14B** — base checkpoint
- **QuestionsGenTrainer.py** — src/ + questions_gen/core/trainer.py
- **stage1-fp-merged** — Hugging Face

Stage 2 · GRPO

RewardCalculator.py

- **Difficulty** — 0.4 difficulty
- **Novelty** — 0.3
- **Rigor** — 0.2
- **Diversity** — 0.1

Optimization

- **GRPO Groups** — 8 problems/group
- **NoveltyConstraint.py** — duplication guard
- **stage2-merged-16bit** — variation generation

Stage 3 · Distill

- **DeepSeekTeacher.py** — DeepSeek-R1
- **Teacher Signals** — difficulty · rigor · innovation · education
- **final-merged-16bit** — Hugging Face

VALIDATION · questions_gen/validation

- **model_validator.py** — single checkpoint
- **batch_validator.py** — algebra · calculus · geometry · number theory
- **quality_evaluator.py** — clarity · difficulty · originality

DEPLOYMENT · interfaces

- **main_cli.py** — validate · batch · quality
- **hf_utils.py** — three published stages
- **ollama_manager.py** — local inference
- **examples/** — quick_demo.py, demo_model_validation.py

SFT, GRPO, and distillation are three model stages. RewardCalculator and NoveltyConstraint belong to GRPO.
Validation and deployment are separate interfaces.